

Introducción a Python

Python fue creado por Guido van Rossum, un programador holandés, a finales de los años 80 y principios de los 90.

Version 1: llamada 0.9.0 -> 1991

Python se diseñó con el objetivo de ser un lenguaje fácil de leer y escribir, con una sintaxis clara y concisa.

Características

- Legibilidad
- Tipado dinámico
- Interpretado
- Multiplataforma
- Amplia biblioteca estándar
- Comunidad activa

Aplicaciones

Se utiliza en una amplia gama de aplicaciones y dominios:

- Desarrollo web
- Ciencia de datos
- Inteligencia artificial y machine learning.
- Automatización de tareas
- Desarrollo de juegos

Conceptos básicos de la sintaxis en Python

Indentación: (espacios o tabulaciones al inicio de una línea) se utiliza para delimitar bloques de código.

if condition:

Bloque de código si la condición es verdadera

instrucción 1

instrucción 2

else

Bloque de código si la condición es falsa

instrucción 3

instrucción 4

Conceptos básicos de la sintaxis en Python

Antes de sumergirnos en conceptos más avanzados, es importante familiarizarse con algunos aspectos básicos de la sintaxis de python.

Indentación

Comentarios

Son líneas de texto en el código que son ignoradas por el intérprete python. Se utilizan para explicar o documentar el código.

Sensibilidad a mayúsculas y minúsculas

Python distingue entre mayúsculas y minúsculas. Por lo tanto, variable, Variable y VARIABLE se consideran variables diferentes.

Punto y Coma

No requiere el uso de punto y coma al final de cada instrucción.

Uso de paréntesis

Se utilizan para agrupar expresiones, definir funciones y realizar llamadas a funciones. por ejemplo:

resultado = (a+b) * c

Fundamentos de Python

Tipos de datos básicos

- **Enteros (int)**: son aquellos que no tienen parte decimal. edad = 25

Cantidad = 100

- **Flotantes (float)**: flotantes también conocidos como número de punto flotante, son aquellos que tienen una parte decimal. precio = 9.99
altura = 1.75

Curso Python

notificación

- **Cadenas de texto (strings)**: Son secuencias de caracteres encerrados entre comillas simples ('...') o dobles ("..."), se utilizan para representar texto.
nombre = "Juan"
mensaje = "¡Hola, mundo!"
- **Booleanos**: Representan los valores de verdad: True (verdadero) y False (falso). Se utilizan comúnmente en expresiones condicionales y operaciones lógicas. por ejemplo: es_mayor_de_edad = True
tiene_descuento = False

↓
Los valores booleanos en python comienzan con una letra mayúscula:
True y False

Variables

Son contenedores que nos permiten almacenar y manipular datos en nuestros programas.

Declaración y Asignación de Variables

Para declarar y asignar un valor a una variable en Python, utilizamos el operador de asignación =. El nombre de la variable va a la izquierda del operador, y el valor que deseas asignar va a la derecha. por ejemplo:

nombre = "Juan"

edad = 25

altura = 1.75

es_estudiante = True

Curso Python

mod/149-0200

Aritméticos

Son los símbolos que permiten realizar operaciones matemáticas sobre números (enteros o flotantes):

- Soma (+): $5 + 3 \rightarrow 8$
- Resta (-): $10 - 4 \rightarrow 6$
- Multiplicación (*): $4 * 3 \rightarrow 12$
- División (/): $10 / 4 \rightarrow 2.5$
(Siempre devuelve un float)
- División entera (//): $10 // 4 \rightarrow 2$ (elimina la parte decimal)
- Módulo o residuo (%): $10 \% 3 \rightarrow 1$ (Útil para saber si un número es par o impar)
- Exponente (**): $2 ** 3 \rightarrow 8$ (2 elevado a la potencia 3)

Operadores de asignación Compuestos: **T**
Combinan una operación con la asignación a la misma variable.

- $x += 5$ equivale a $x = x + 5$
- $x *= 2$ equivale a $x = x * 2$

Estructuras de Control

Controlan el flujo de ejecución del programa según condiciones o repeticiones.

Condicionales (if, elif, else)

Evalúan expresiones booleanas (**True o False**).

Python

```
edad = 18
```

```
if edad >= 18:
```

```
    print("Mayor de edad")
```

```
elif edad >= 13:
```

```
    print("Adolescente")
```

```
else: print("Menor de edad")
```

Curso Python / nupll nmax

Operadores de Comparación

`==` (Igual al),

`!=` (Diferente de)

`>`, `<`, `>=`, `<=`

Operadores lógicos:

- `and`: Ambas Condiciones deben de ser verdaderas.
- `or`: Al menos una condición debe de ser verdadera.
- `not`: Invierte el valor de verdad.

Bucles / Loops

Permiten repetir la ejecución de un bloque de código.

Bucle `for`

Ideal para iterar sobre una secuencia (listas, tuplas, cadenas, rangos).

Python

Imprimir del 0 al 4 usando `range()`

```
i in range(5):
```

```
    print(i)
```

Bucle `While`

Se ejecutan mientras una condición siga siendo verdadera.

Python

```
contador = 0
```

```
while contador < 3:
```

```
    print(contador)
```

```
    contador += 1
```

Control de bucles:

- `break`: interrumpe el bucle por completo.
- `Continue`: Salta el resto de la iteración actual y pasa a la siguiente.

Curso Python

Estructuras de datos

Colecciones que permiten almacenar múltiples valores en una sola variable.

Tuplas (tuple)

Ordenados e inmutables (no se pueden modificar, agregar o eliminar elementos tras crearse).

Uso de paréntesis `()`

Diccionarios (dict)

Estructuras de clave-valor (key: value). No admiten claves duplicadas.

Uso de llaves `{}`.

Conjuntos (set)

Colecciones no ordenadas de elementos únicos (eliminan duplicados automáticamente).

Uso de llaves `{}` o la función `set()`.

Python

```
numeros = {1, 2, 2, 3, 4}
```

```
print(numeros) # imprime {1, 2, 3, 4}
```

Operaciones comunes de conjuntos.

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a.union(b)) # {1, 2, 3, 4, 5}
```

```
print(a.intersection(b)) # {3}
```